

DSA Algorithm Documentation

Real-Graph Analysis, Correctness Evidence & Performance Experiment

Prepared by: Dzifa

This report presents evidence for the implemented Dijkstra, Prim and Kruskal algorithms using the project's real 65-location graph, plus path reconstruction traces and the completed runtime experiment.

1. Algorithms Implemented

Dijkstra: shortest paths from a source in a weighted graph; stores distances and predecessors and reconstructs routes. Expected complexity: $O((V + E) \log V)$.

Prim: minimum spanning tree using the lowest connecting edge. Expected complexity: $O(E \log V)$.

Kruskal: minimum spanning tree using sorted edges and DisjointSet cycle prevention. Expected complexity: $O(E \log E)$.

2. Real 65-Location Graph

The real dataset contains 65 locations (L001–L065) and 110 road records. Road records contain distance, travel time and condition-weight fields; the project Graph represents the roads as positive-weight undirected edges.

3. Dijkstra on the Real Graph

Dijkstra was run from L001. Selected distance and predecessor evidence is shown below.

Location	Distance	Predecessor	Path
L001	0.00 km	—	L001
L065	40.45 km	L064	L001 → L002 → L015 → L045 → ... → L064 → L065
L046	5.10 km	L045	L001 → L002 → L015 → L045 → L046
L007	0.35 km	L001	L001 → L007
L053	19.45 km	L052	L001 → L002 → L015 → L045 → ... → L052 → L053

3.1 Example Shortest Paths

Route	Reconstructed path	Distance
L001 → L046	L001 → L002 → L015 → L045 → L046	5.10 km
L001 → L053	L001 → L002 → L015 → L045 → L046 → L047 → L048 → L049 → L050 → L051 → L052 → L053	19.45 km
L001 → L065	L001 → L002 → L015 → L045 → L046 → L047 → L048 → L049 → L050 → L051 → L052 → L053 → L054 → L055 → L056 → L057 → L058 → L059 → L060 → L061 → L062 → L063 → L064 → L065	40.45 km

4. Dijkstra Path-Reconstruction Trace Tables

getPath() follows predecessor links backwards from the destination until the source is reached, then produces the route in source-to-destination order.

Trace Table 1 — L001 → L046

Step	Current	Previous	Action
1	L046	L045	Move to L045
2	L045	L015	Move to L015
3	L015	L002	Move to L002
4	L002	L001	Move to L001
5	L001	-1	Source reached; stop

Final path: L001 → L002 → L015 → L045 → L046 **Distance:** 5.10 km

Trace Table 2 — L001 → L053

Step	Current	Previous	Action
1	L053	L052	Move to L052
2	L052	L051	Move to L051
3	L051	L050	Move to L050
4	L050	L049	Move to L049
5	L049	L048	Move to L048
6	L048	L047	Move to L047
7	L047	L046	Move to L046
8	L046	L045	Move to L045
9	L045	L015	Move to L015
10	L015	L002	Move to L002
11	L002	L001	Move to L001
12	L001	-1	Source reached; stop

Final path: L001 → L002 → L015 → L045 → L046 → L047 → L048 → L049 → L050 → L051 → L052 → L053 **Distance:** 19.45 km

5. Prim & Kruskal on the Real Graph

Algorithm	MST edges	Total weight
Prim	64	79.70 km
Kruskal	64	79.70 km

Correctness evidence: Both algorithms produced 64 edges for the 65-vertex graph and exactly the same MST total, 79.70 km. This matching independent result is strong correctness evidence.

6. Reversed Comparator Explanation

The project's Heap is a max-heap for scheduling and urgency operations. Dijkstra needs the smallest tentative distance first, so Entry.compareTo() is deliberately reversed, making the max-heap behave as a min-priority queue for Dijkstra.

7. Performance Experiment

The benchmark ran three times at graph sizes 50, 100, 200 and 500 vertices. Runtime was recorded in nanoseconds and saved to `mst_runtime.csv`.

Vertices	Prim 1	Kruskal 1	Prim 2	Kruskal 2	Prim 3	Kruskal 3
50	665,300	459,100	453,700	509,600	190,300	218,500
100	794,700	417,600	635,500	764,500	420,700	1,078,700
200	699,700	1,303,500	1,117,000	1,722,600	761,100	1,233,500
500	2,461,100	2,829,100	2,601,200	3,620,100	2,136,300	2,002,400

The measurements generally increase with graph size, while individual runs vary because of JVM warm-up and system conditions. The experiment is empirical evidence and complements, rather than replaces, asymptotic analysis.

8. Complexity Analysis

Algorithm	Expected complexity	Supporting structure
Dijkstra	$O((V + E) \log V)$	Graph + Heap priority queue
Prim	$O(E \log V)$	Graph + Heap priority queue
Kruskal	$O(E \log E)$	Graph + DisjointSet + sorting

9. Conclusion

The real-graph analysis demonstrates successful shortest-path reconstruction from L001 and matching MST costs from Prim and Kruskal. The two MST implementations each returned 64 edges with total weight 79.70 km. The reversed comparator correctly adapts the project's max-heap to Dijkstra's minimum-distance requirement. The benchmark adds empirical runtime evidence across increasing graph sizes.